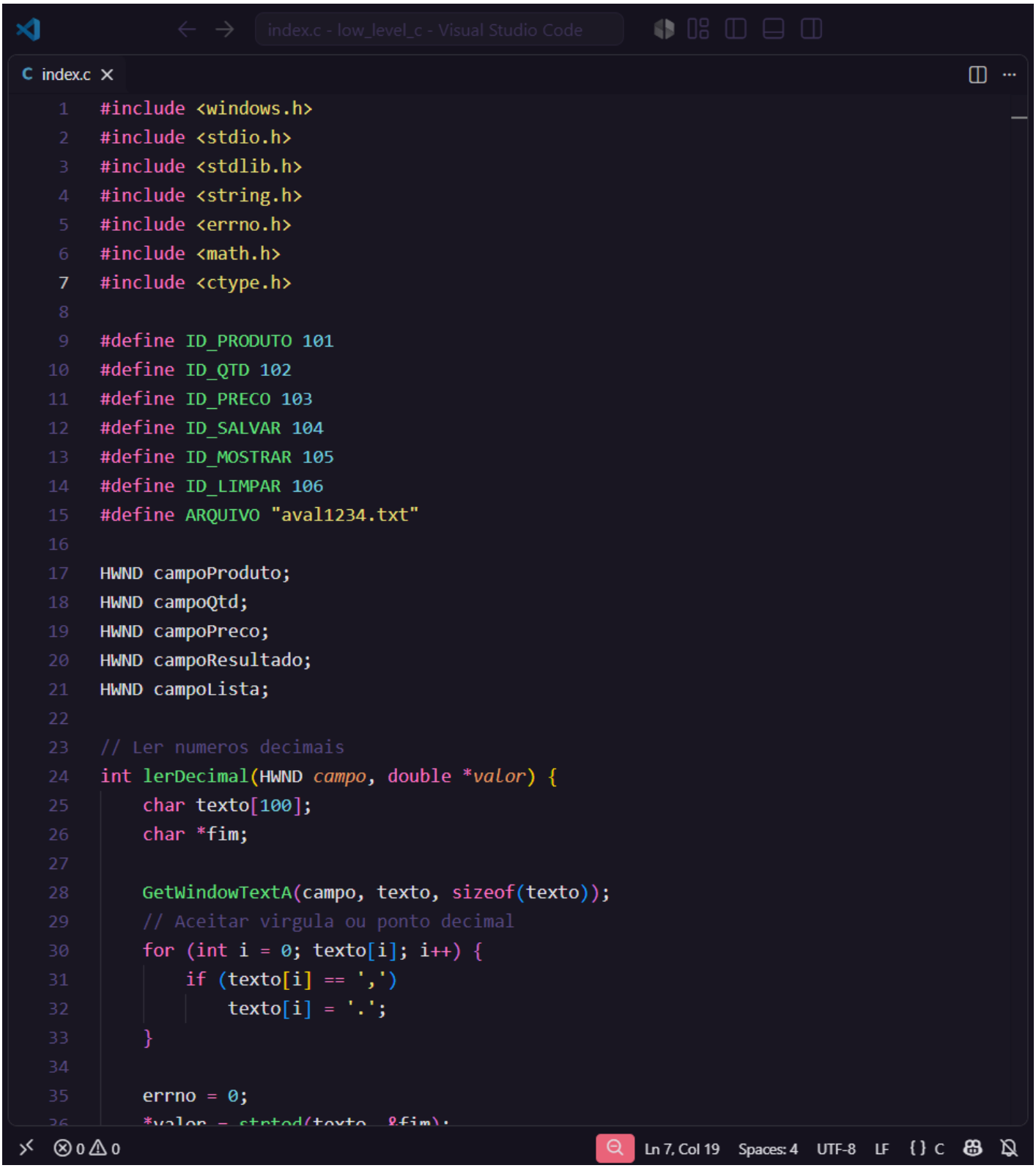
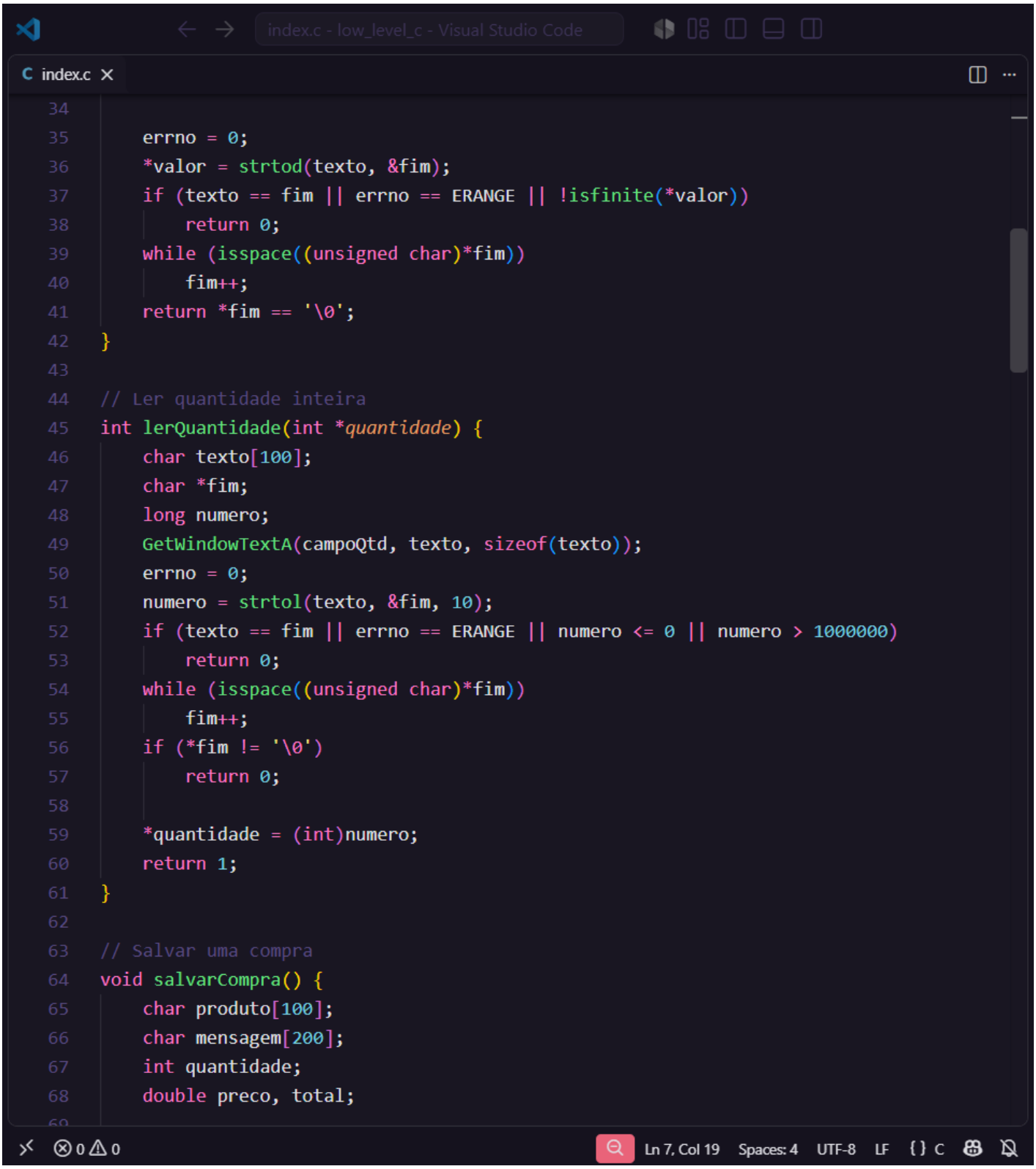
PROGRAMAÇÃO DE SOFTWARE BÁSICO EM C

Aluno: Carlos Eduardo Silva Ferreira
Matrícula: 202408542283
Professor: Alessandro Calin

Repositório GitHub: https://github.com/Carloseduardo-dev/low_level_c.git

1 - Insira uma captura de tela (print) do código-fonte completo digitado no ambiente de desenvolvimento:



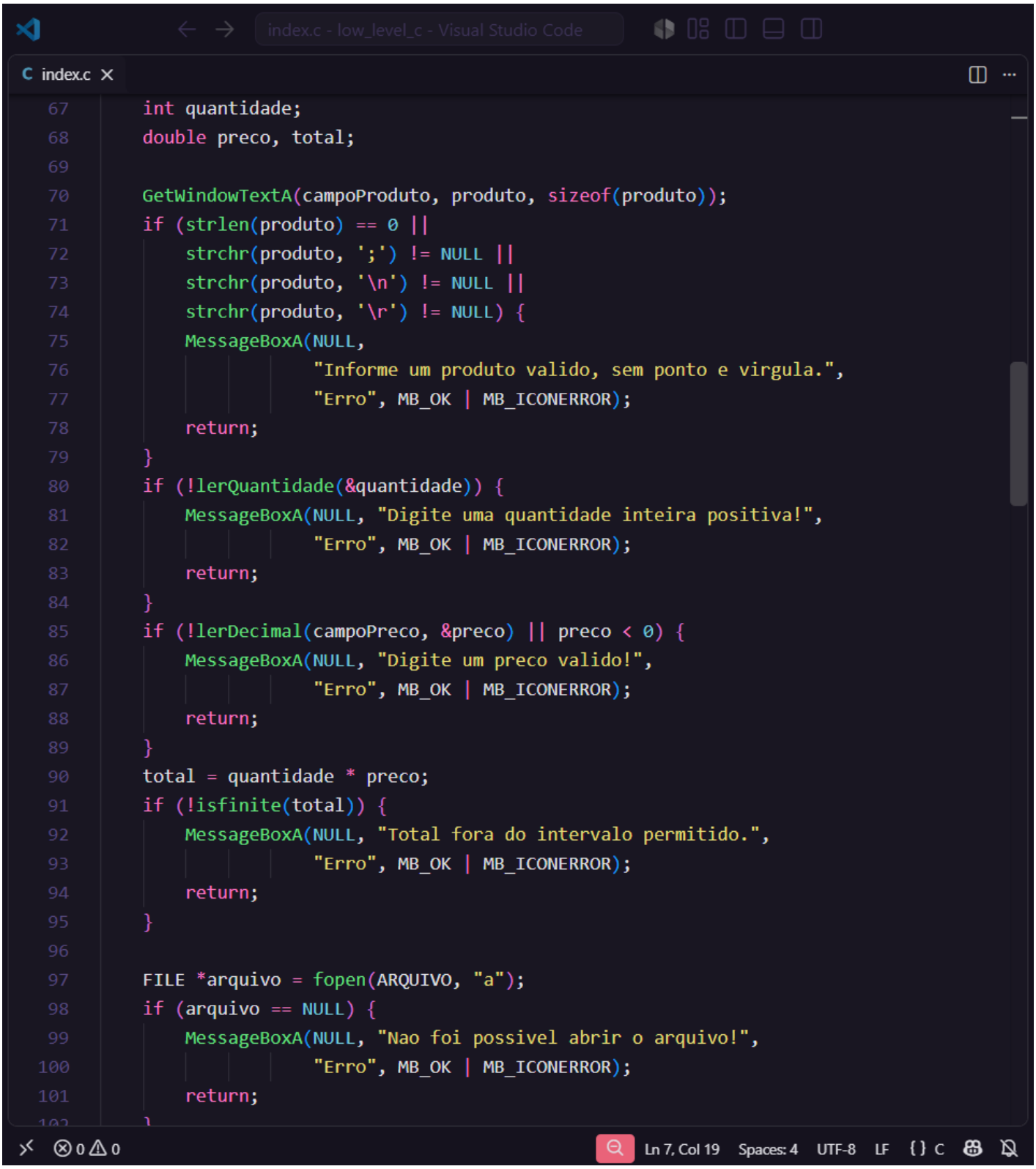


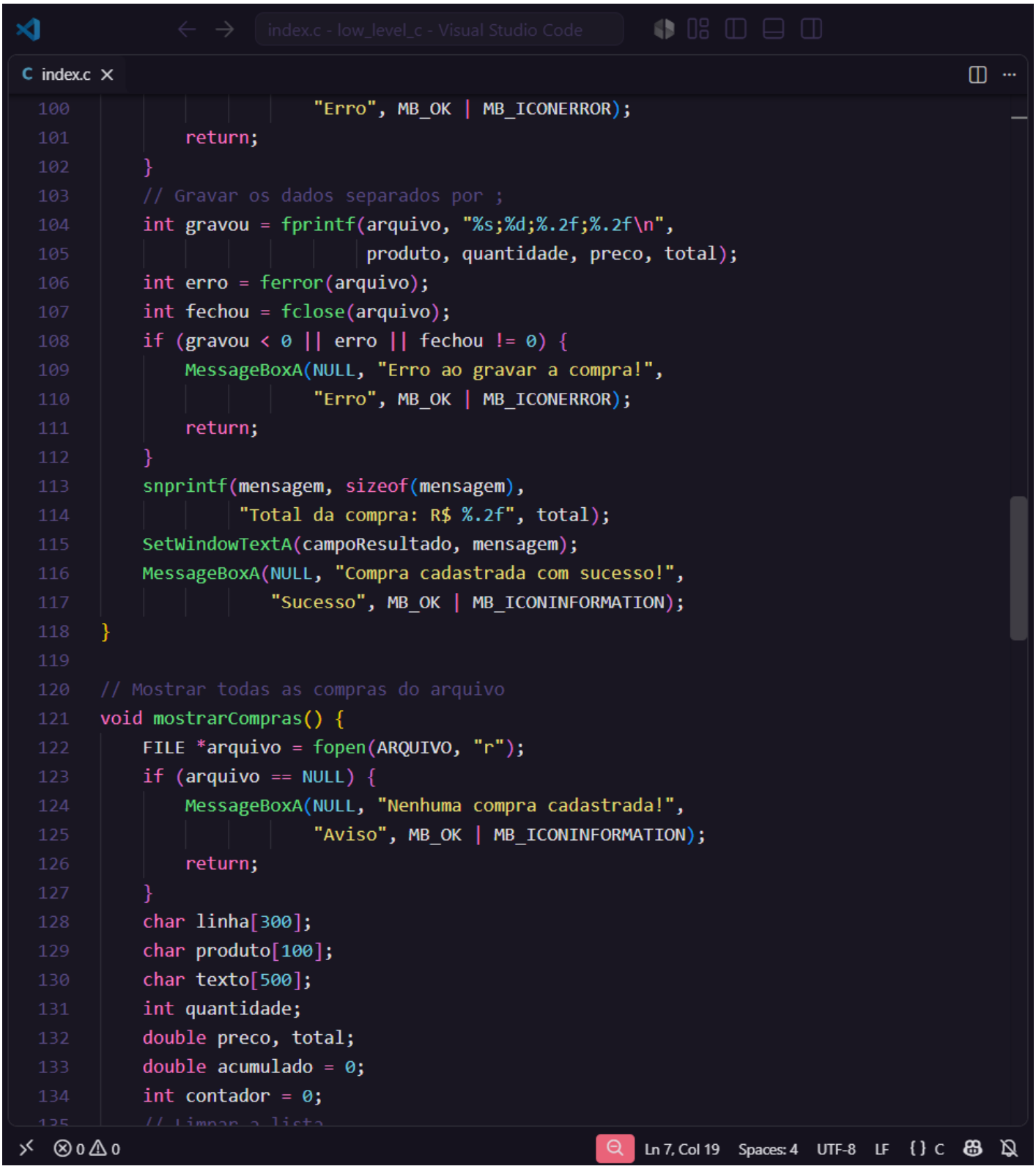

```c
    errno = 0;
    *valor = strtod(texto, &fim);
    if (texto == fim || errno == ERANGE || !isfinite(*valor))
        return 0;
    while (isspace((unsigned char)*fim))
        fim++;
    return *fim == '\0';
}

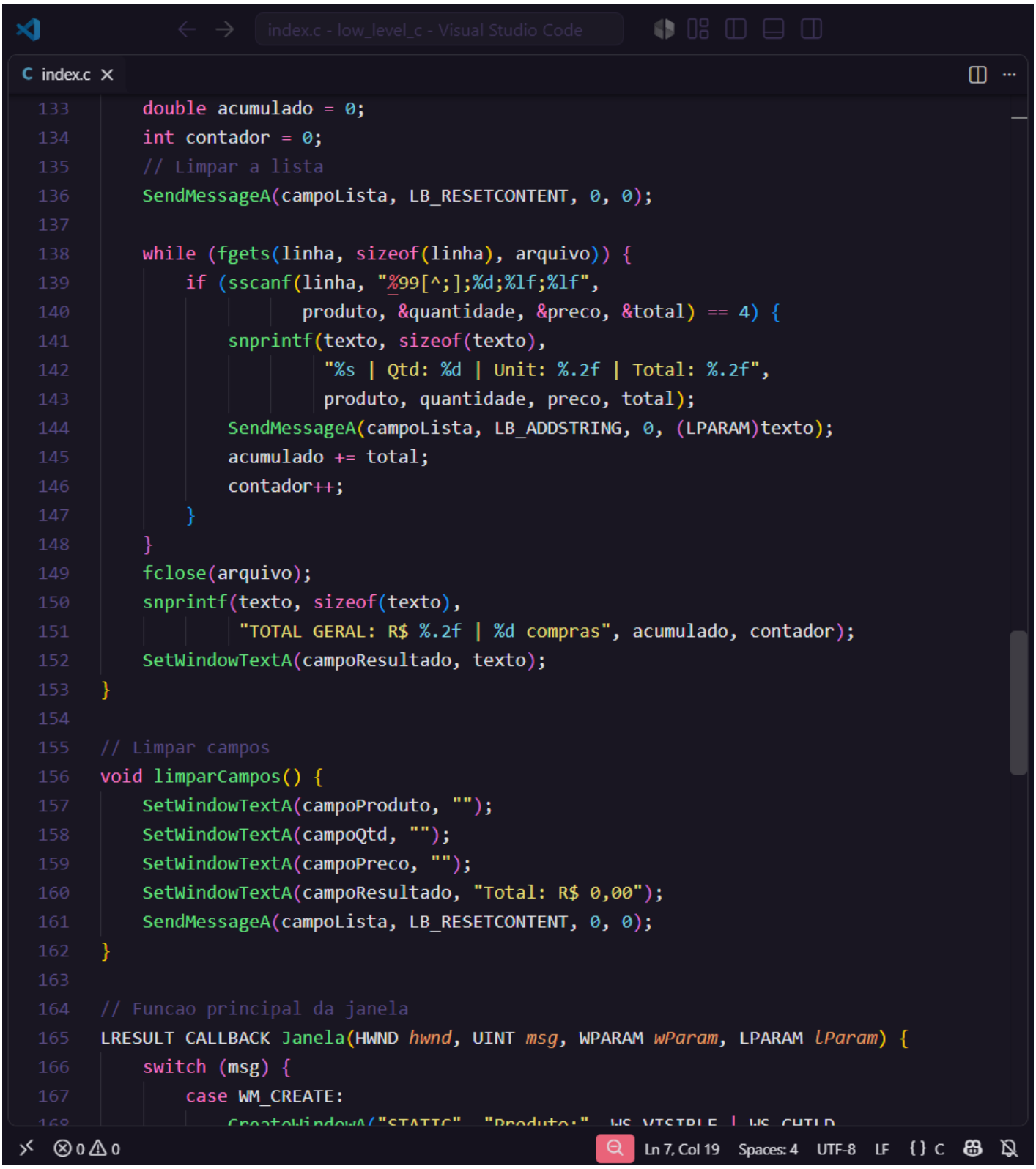
// Ler quantidade inteira
int lerQuantidade(int *quantidade) {
    char texto[100];
    char *fim;
    long numero;
    GetWindowTextA(campoQtd, texto, sizeof(texto));
    errno = 0;
    numero = strtol(texto, &fim, 10);
    if (texto == fim || errno == ERANGE || numero <= 0 || numero > 1000000)
        return 0;
    while (isspace((unsigned char)*fim))
        fim++;
    if (*fim != '\0')
        return 0;

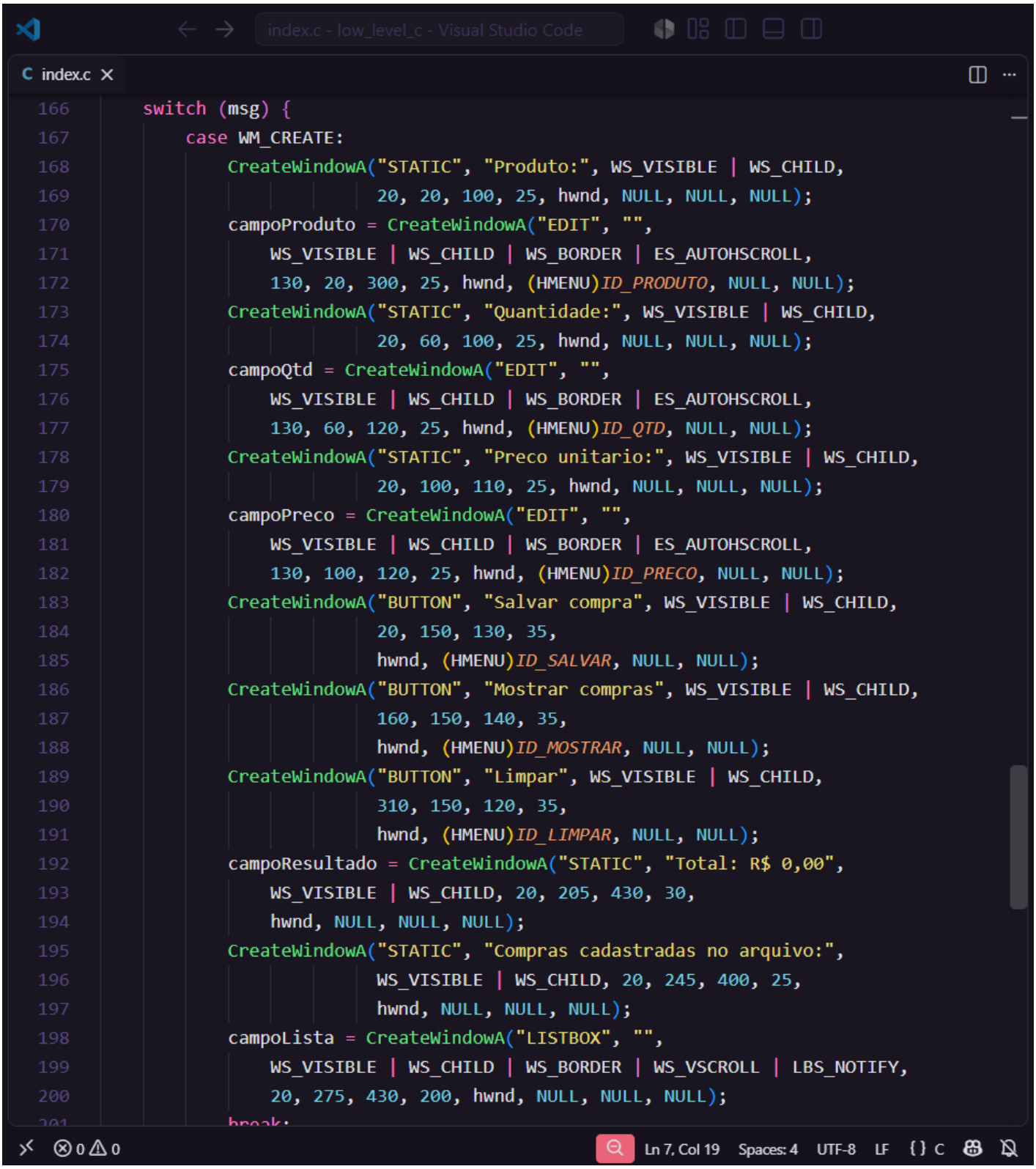
    *quantidade = (int)numero;
    return 1;
}

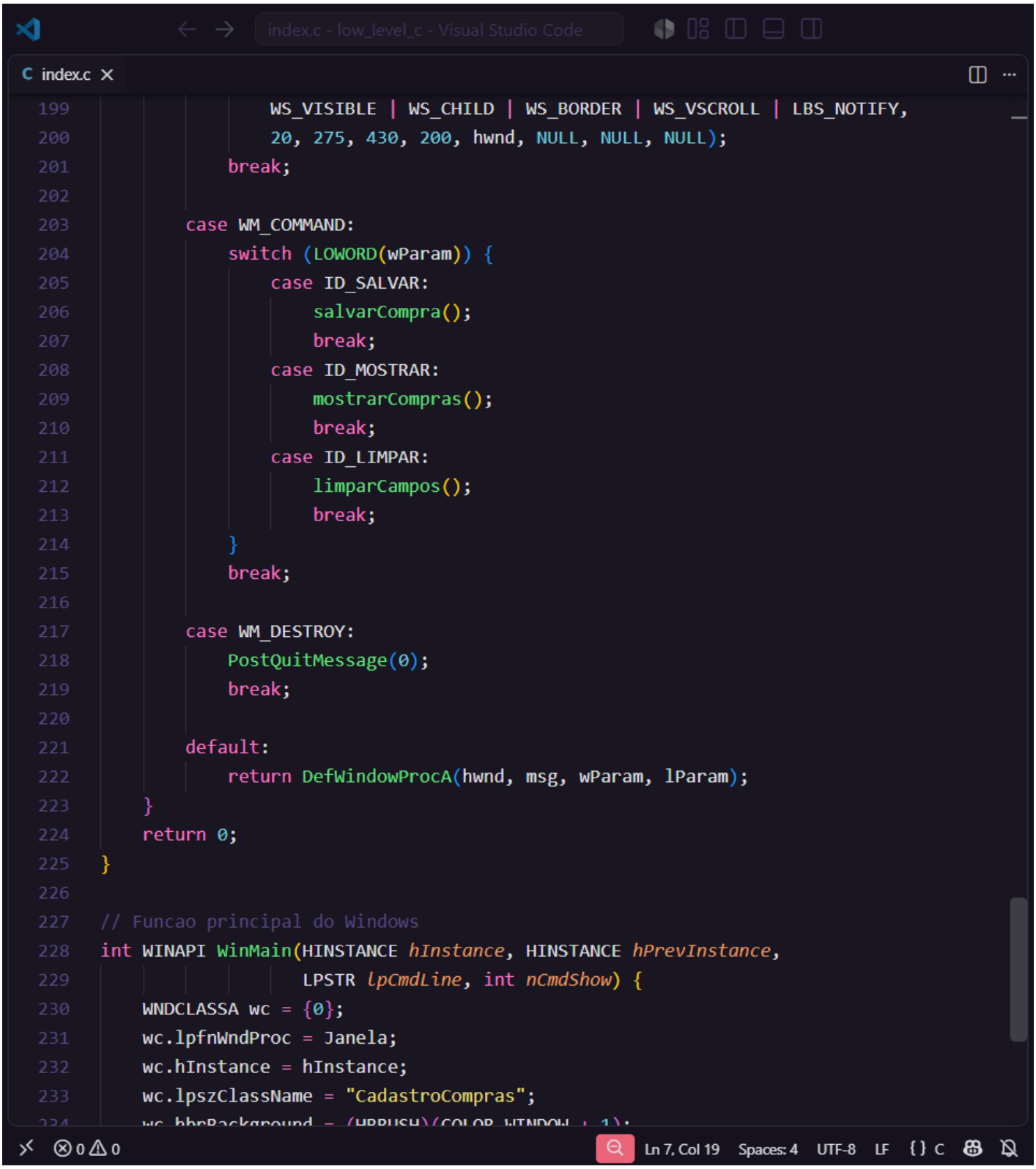
// Salvar uma compra
void salvarCompra() {
    char produto[100];
    char mensagem[200];
    int quantidade;
    double preco, total;
```

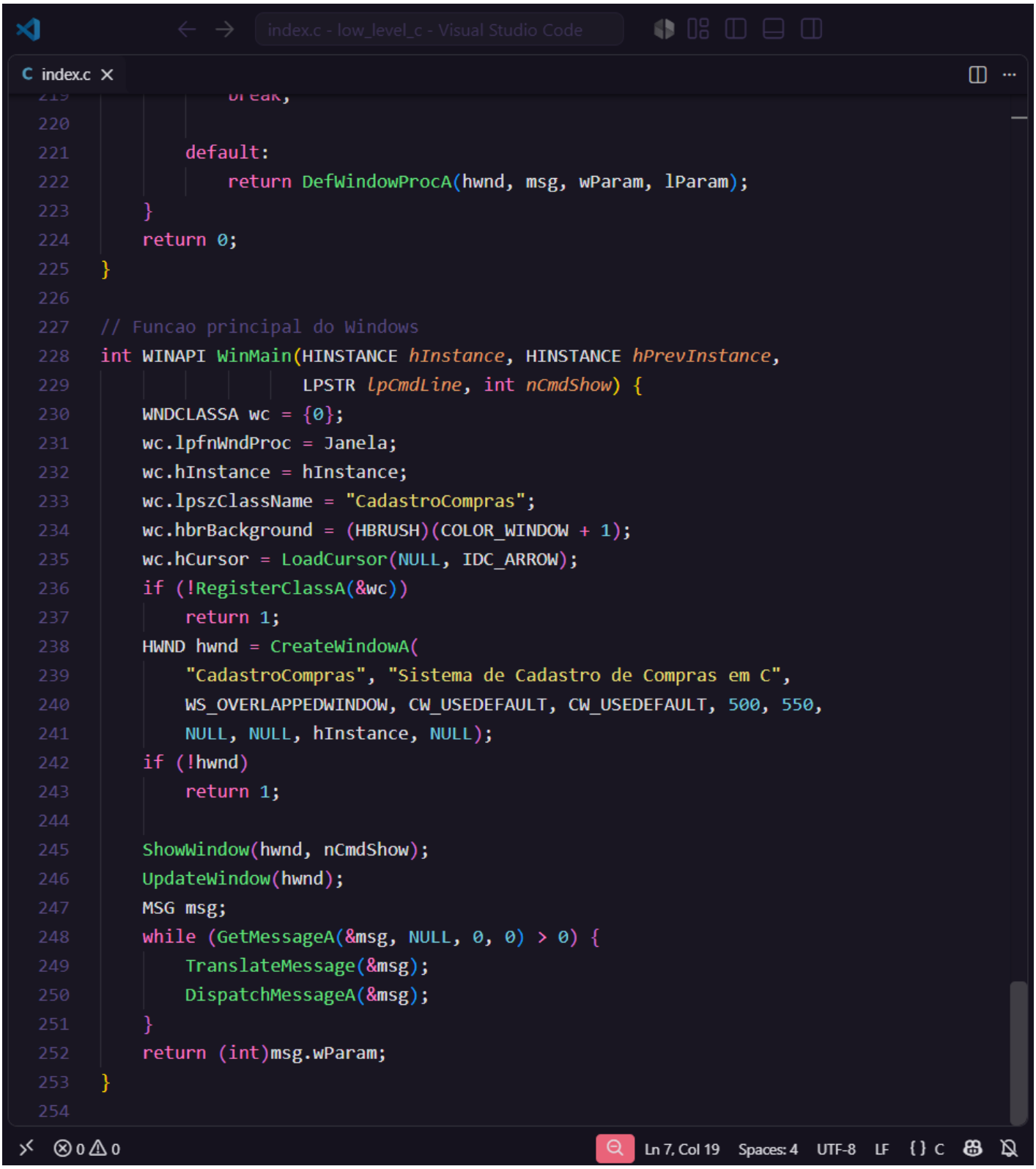

index.c - low_level_c - Visual Studio Code
index.c
int quantidade;
double preco, total;
GetWindowTextA(campoProduto, produto, sizeof(produto));
if (strlen(produto) == 0 ||
strchr(produto, ';') != NULL ||
strchr(produto, '\n') != NULL ||
strchr(produto, '\r') != NULL) {
MessageBoxA(NULL,
"Informe um produto valido, sem ponto e virgula.",
"Erro", MB_OK | MB_ICONERROR);
return;
}
if (!lerQuantidade(&quantidade)) {
MessageBoxA(NULL, "Digite uma quantidade inteira positiva!",
"Erro", MB_OK | MB_ICONERROR);
return;
}
if (!lerDecimal(campoPreco, &preco) || preco < 0) {
MessageBoxA(NULL, "Digite um preco valido!",
"Erro", MB_OK | MB_ICONERROR);
return;
}
total = quantidade * preco;
if (!isfinite(total)) {
MessageBoxA(NULL, "Total fora do intervalo permitido.",
"Erro", MB_OK | MB_ICONERROR);
return;
}
FILE *arquivo = fopen(ARQUIVO, "a");
if (arquivo == NULL) {
MessageBoxA(NULL, "Nao foi possivel abrir o arquivo!",
"Erro", MB_OK | MB_ICONERROR);
return;
}
0 0
Ln 7, Col 19
Spaces: 4
UTF-8
LF
{} C


index.c - low_level_c - Visual Studio Code
index.c
"Erro", MB_OK | MB_ICONERROR);
return;
}
// Gravar os dados separados por ;
int gravou = fprintf(arquivo, "%s;%d;%.2f;%.2f\n",
produto, quantidade, preco, total);
int erro = ferror(arquivo);
int fechou = fclose(arquivo);
if (gravou < 0 || erro || fechou != 0) {
MessageBoxA(NULL, "Erro ao gravar a compra!",
"Erro", MB_OK | MB_ICONERROR);
return;
}
snprintf(mensagem, sizeof(mensagem),
"Total da compra: R$ %.2f", total);
SetWindowTextA(campoResultado, mensagem);
MessageBoxA(NULL, "Compra cadastrada com sucesso!",
"Sucesso", MB_OK | MB_ICONINFORMATION);
}
// Mostrar todas as compras do arquivo
void mostrarCompras() {
FILE *arquivo = fopen(ARQUIVO, "r");
if (arquivo == NULL) {
MessageBoxA(NULL, "Nenhuma compra cadastrada!",
"Aviso", MB_OK | MB_ICONINFORMATION);
return;
}
char linha[300];
char produto[100];
char texto[500];
int quantidade;
double preco, total;
double acumulado = 0;
int contador = 0;
0 0
Ln 7, Col 19
Spaces: 4
UTF-8
LF
{} C


index.c - low_level_c - Visual Studio Code
index.c
double acumulado = 0;
int contador = 0;
// Limpar a lista
SendMessageA(campoLista, LB_RESETCONTENT, 0, 0);
while (fgets(linha, sizeof(linha), arquivo)) {
if (sscanf(linha, "%99[^;];%d;%lf;%lf",
produto, &quantidade, &preco, &total) == 4) {
snprintf(texto, sizeof(texto),
"%s | Qtd: %d | Unit: %.2f | Total: %.2f",
produto, quantidade, preco, total);
SendMessageA(campoLista, LB_ADDSTRING, 0, (LPARAM)texto);
acumulado += total;
contador++;
}
}
fclose(arquivo);
snprintf(texto, sizeof(texto),
"TOTAL GERAL: R$ %.2f | %d compras", acumulado, contador);
SetWindowTextA(campoResultado, texto);
}
// Limpar campos
void limparCampos() {
SetWindowTextA(campoProduto, "");
SetWindowTextA(campoQtd, "");
SetWindowTextA(campoPreco, "");
SetWindowTextA(campoResultado, "Total: R$ 0,00");
SendMessageA(campoLista, LB_RESETCONTENT, 0, 0);
}
// Funcao principal da janela
LRESULT CALLBACK Janela(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
switch (msg) {
case WM_CREATE:
Ln 7, Col 19
Spaces: 4
UTF-8
LF
C


index.c - low_level_c - Visual Studio Code
index.c
switch (msg) {
case WM_CREATE:
CreateWindowA("STATIC", "Produto:", WS_VISIBLE | WS_CHILD,
20, 20, 100, 25, hwnd, NULL, NULL, NULL);
campoProduto = CreateWindowA("EDIT", "",
WS_VISIBLE | WS_CHILD | WS_BORDER | ES_AUTOHSCROLL,
130, 20, 300, 25, hwnd, (HMENU)ID_PRODUTO, NULL, NULL);
CreateWindowA("STATIC", "Quantidade:", WS_VISIBLE | WS_CHILD,
20, 60, 100, 25, hwnd, NULL, NULL, NULL);
campoQtd = CreateWindowA("EDIT", "",
WS_VISIBLE | WS_CHILD | WS_BORDER | ES_AUTOHSCROLL,
130, 60, 120, 25, hwnd, (HMENU)ID_QTD, NULL, NULL);
CreateWindowA("STATIC", "Preco unitario:", WS_VISIBLE | WS_CHILD,
20, 100, 110, 25, hwnd, NULL, NULL, NULL);
campoPreco = CreateWindowA("EDIT", "",
WS_VISIBLE | WS_CHILD | WS_BORDER | ES_AUTOHSCROLL,
130, 100, 120, 25, hwnd, (HMENU)ID_PRECO, NULL, NULL);
CreateWindowA("BUTTON", "Salvar compra", WS_VISIBLE | WS_CHILD,
20, 150, 130, 35,
hwnd, (HMENU)ID_SALVAR, NULL, NULL);
CreateWindowA("BUTTON", "Mostrar compras", WS_VISIBLE | WS_CHILD,
160, 150, 140, 35,
hwnd, (HMENU)ID_MOSTRAR, NULL, NULL);
CreateWindowA("BUTTON", "Limpar", WS_VISIBLE | WS_CHILD,
310, 150, 120, 35,
hwnd, (HMENU)ID_LIMPAR, NULL, NULL);
campoResultado = CreateWindowA("STATIC", "Total: R$ 0,00",
WS_VISIBLE | WS_CHILD, 20, 205, 430, 30,
hwnd, NULL, NULL, NULL);
CreateWindowA("STATIC", "Compras cadastradas no arquivo:",
WS_VISIBLE | WS_CHILD, 20, 245, 400, 25,
hwnd, NULL, NULL, NULL);
campoLista = CreateWindowA("LISTBOX", "",
WS_VISIBLE | WS_CHILD | WS_BORDER | WS_VSCROLL | LBS_NOTIFY,
20, 275, 430, 200, hwnd, NULL, NULL, NULL);
break;
0 0
Ln 7, Col 19
Spaces: 4
UTF-8
LF
{} C


index.c - low_level_c - Visual Studio Code
index.c
199 WS_VISIBLE | WS_CHILD | WS_BORDER | WS_VSCROLL | LBS_NOTIFY,
200 20, 275, 430, 200, hwnd, NULL, NULL, NULL);
201 break;
202
203 case WM_COMMAND:
204 switch (LOWORD(wParam)) {
205 case ID_SALVAR:
206 salvarCompra();
207 break;
208 case ID_MOSTRAR:
209 mostrarCompras();
210 break;
211 case ID_LIMPAR:
212 limparCampos();
213 break;
214 }
215 break;
216
217 case WM_DESTROY:
218 PostQuitMessage(0);
219 break;
220
221 default:
222 return DefWindowProcA(hwnd, msg, wParam, lParam);
223 }
224 return 0;
225 }
226
227 // Funcao principal do Windows
228 int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
229 LPSTR lpCmdLine, int nCmdShow) {
230 WNDCLASSA wc = {0};
231 wc.lpfnWndProc = Janela;
232 wc.hInstance = hInstance;
233 wc.lpszClassName = "CadastroCompras";
0 0
Ln 7, Col 19
Spaces: 4
UTF-8
LF
{} C


index.c - low_level_c - Visual Studio Code
index.c
break;
default:
return DefWindowProcA(hwnd, msg, wParam, lParam);
}
return 0;
}
// Funcao principal do Windows
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow) {
WNDCLASSA wc = {0};
wc.lpfnWndProc = Janela;
wc.hInstance = hInstance;
wc.lpszClassName = "CadastroCompras";
wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);
wc.hCursor = LoadCursor(NULL, IDC_ARROW);
if (!RegisterClassA(&wc))
return 1;
HWND hwnd = CreateWindowA(
"CadastroCompras", "Sistema de Cadastro de Compras em C",
WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 500, 550,
NULL, NULL, hInstance, NULL);
if (!hwnd)
return 1;
ShowWindow(hwnd, nCmdShow);
UpdateWindow(hwnd);
MSG msg;
while (GetMessageA(&msg, NULL, 0, 0) > 0) {
TranslateMessage(&msg);
DispatchMessageA(&msg);
}
return (int)msg.wParam;
}
0 0
Ln 7, Col 19
Spaces: 4
UTF-8
LF
{} C

2 - Insira uma captura de tela (print) do programa em execução, demonstrando seu funcionamento com pelo menos cinco inserções de dados, incluindo os resultados apresentados pelo sistema.




Sistema de Cadastro de Compras em C
Produto:
Mouse
Quantidade:
3
Preco unitario:
50,00
Salvar compra
Mostrar compras
Limpar
Total da compra: R$ 150.00
Compras cadastradas no arquivo:


Sistema de Cadastro de Compras em C
Produto:
Monitor
Quantidade:
1
Preco unitario:
800.00
Salvar compra
Mostrar compras
Limpar
Total da compra: R$ 800.00
Compras cadastradas no arquivo:


Sistema de Cadastro de Compras em C
Produto:
Headset
Quantidade:
4
Preco unitario:
75,50
Salvar compra
Mostrar compras
Limpar
Total da compra: R$ 302.00
Compras cadastradas no arquivo:


Sistema de Cadastro de Compras em C
Produto:
Webcam
Quantidade:
2
Preco unitario:
125.90
Salvar compra
Mostrar compras
Limpar
Total da compra: R$ 251.80
Compras cadastradas no arquivo:


Sistema de Cadastro de Compras em C
Produto:
Webcam
Quantidade:
2
Preco unitario:
125.90
Salvar compra
Mostrar compras
Limpar
TOTAL GERAL: R$ 1703.80 | 5 compras
Compras cadastradas no arquivo:
Teclado | Qtd: 2 | Unit: 100.00 | Total: 200.00
Mouse | Qtd: 3 | Unit: 50.00 | Total: 150.00
Monitor | Qtd: 1 | Unit: 800.00 | Total: 800.00
Headset | Qtd: 4 | Unit: 75.50 | Total: 302.00
Webcam | Qtd: 2 | Unit: 125.90 | Total: 251.80

Questões objetivas

Questão 1

Alternativa B) Permitir a criação e o gerenciamento de janelas, botões e outros componentes gráficos do Windows.

Questão 2

Alternativa C) Criar um campo de texto que permite a entrada de dados pelo usuário.

Questão 3

Alternativa D) Atribuir identificadores aos componentes para permitir seu reconhecimento durante o processamento de eventos.

Questão 4

Alternativa A) O conteúdo digitado no campo gráfico é copiado para a variável produto.

Questão 5

Alternativa E) Rejeitar valores inválidos, fora do intervalo permitido ou quantidades não positivas.

Questão 6

Alternativa B) R$ 302,00.

Questão 7

Alternativa C) Abre o arquivo para acrescentar registros ao final, criando-o caso não exista.

Questão 8

Alternativa A) Separar os campos do registro para facilitar sua posterior leitura e interpretação.

Questão 9

Alternativa D) Ler sucessivamente as linhas do arquivo enquanto houver dados disponíveis.

Questão 10

Alternativa E) Extrair os campos de uma linha de texto e armazená-los nas variáveis correspondentes.

Questão 11

Alternativa B) Adiciona uma nova linha de texto ao componente gráfico de lista.

Questão 12

Alternativa C) R$ 1.150,00 e 3.

Questão 13

Explique o funcionamento da função desde a obtenção dos dados na interface até o armazenamento em aval1234.txt, abordando validação, cálculo do total e manipulação do arquivo.

Obtenção do produto. A função declara os vetores produto e mensagem, a variável inteira quantidade e as variáveis double preco e total. GetWindowTextA(campoProduto, produto, sizeof(produto)) copia o texto do campo gráfico para o vetor produto, limitado pelo tamanho desse vetor.

Validação dos dados. O código usa strlen para rejeitar um nome vazio e strchr para impedir ponto e vírgula, quebra de linha e retorno de carro no produto. Isso evita que esses caracteres interfiram na separação dos campos e dos registros no arquivo. Se houver erro, MessageBoxA informa o problema e return encerra a tentativa de cadastro antes da gravação.

A função lerQuantidade lê o campo de quantidade com GetWindowTextA e converte o texto com strtol. Ela verifica se houve conversão, se ocorreu ERANGE e se o valor está entre 1 e 1.000.000. Depois ignora espaços finais e rejeita outros caracteres restantes. Somente uma quantidade válida é atribuída à variável recebida por ponteiro.

A função lerDecimal lê o preço, substitui vírgulas por pontos e utiliza strtod para convertê-lo em double. Ela rejeita ausência de conversão, ERANGE, valores não finitos e caracteres inválidos após o número. Em salvarCompra, preco < 0 também é rejeitado. O valor zero é aceito pelo código do enunciado.

Cálculo do total. Com os dados válidos, total = quantidade * preco calcula o valor da compra. A condição !isfinite(total) impede continuar se o resultado não for finito. Por exemplo, quatro unidades a R$ 75,50 resultam em R$ 302,00.

Gravação. fopen(ARQUIVO, "a") abre aval1234.txt para acrescentar dados ao final, criando o arquivo se necessário. Se retornar NULL, a função apresenta uma mensagem e termina. fprintf grava produto, quantidade, preço e total separados por ponto e vírgula, com duas casas decimais para os valores e uma quebra de linha no final.

O retorno de fprintf é guardado em gravou; ferror consulta erros de escrita e fclose fecha o arquivo e descarrega os dados pendentes. O código verifica as três condições para informar falha de gravação. Se tudo ocorrer corretamente, snprintf formata o total da compra, SetWindowTextA atualiza campoResultado e MessageBoxA confirma o cadastro. A função não limpa os campos automaticamente.

Questão 14

Explique a leitura e a apresentação das compras, descrevendo fopen(), fgets(), sscanf(), SendMessageA(), fclose() e o cálculo do valor acumulado.

Abertura para leitura. Ao clicar em Mostrar compras, o evento WM_COMMAND identifica ID_MOSTRAR e chama mostrarCompras(). A função usa fopen(ARQUIVO, "r") para abrir aval1234.txt em modo de leitura. Caso a abertura falhe, apresenta a mensagem "Nenhuma compra cadastrada!" e retorna, sem tentar acessar um ponteiro de arquivo inválido.

Preparação. São declarados os vetores linha, produto e texto, as variáveis quantidade, preco e total, além de acumulado e contador, ambos iniciados em zero. SendMessageA(campoLista, LB_RESETCONTENT, 0, 0) limpa a lista exibida antes de preenchê-la novamente. Assim, consultar o arquivo repetidamente não duplica os itens apresentados.

Leitura das linhas. O laço while (fgets(linha, sizeof(linha), arquivo)) lê uma linha de cada vez para o vetor linha, respeitando seu tamanho. Enquanto fgets retorna um ponteiro não nulo, o conteúdo é processado. Ao chegar ao fim do arquivo, ou em caso de falha de leitura, o laço termina.

Extração dos campos. sscanf interpreta cada linha com o formato "%99[^;];%d;%lf;%lf". O primeiro especificador lê até 99 caracteres do produto antes do ponto e vírgula; %d recupera a quantidade e os dois %lf recuperam preço e total em variáveis double. Os endereços dessas variáveis são passados com &. Somente quando o retorno é igual a 4 o registro é incluído na lista e no cálculo.

Apresentação e acumulação. snprintf monta uma descrição com nome do produto, quantidade, preço unitário e total. SendMessageA(campoLista, LB_ADDSTRING, 0, (LPARAM)texto) acrescenta essa descrição ao componente LISTBOX. Para cada registro interpretado, acumulado += total soma o valor da compra e contador++ aumenta em uma unidade o número de compras, independentemente da quantidade de produtos em cada registro.

Finalização. Depois do laço, fclose(arquivo) fecha o arquivo aberto para leitura. snprintf prepara a mensagem "TOTAL GERAL: R$ ... | ... compras" e SetWindowTextA(campoResultado, texto) a apresenta na janela. Com Teclado de R$ 200,00, Mouse de R$ 150,00 e Monitor de R$ 800,00, o acumulado é R$ 1.150,00 e o contador é 3. Com as cinco inserções deste relatório, o resultado é R$ 1.703,80 e 5 compras.